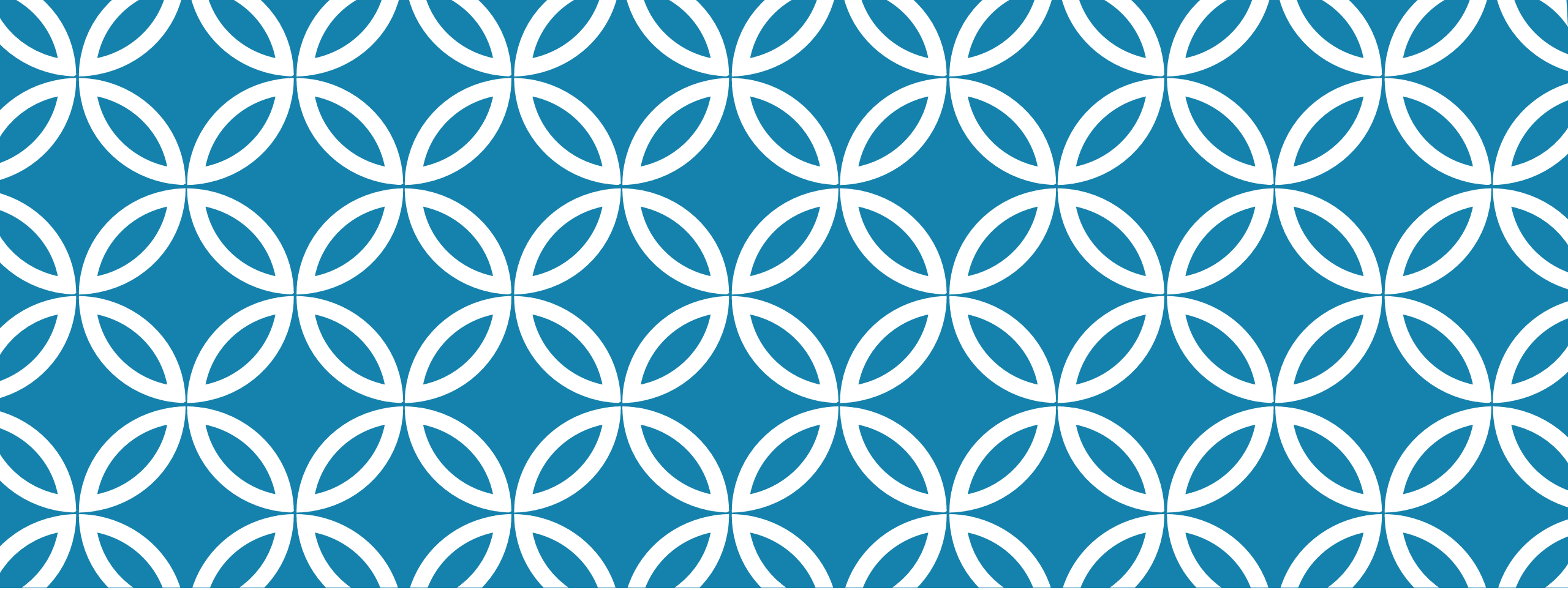


GÖRSEL PROGRAMLAMA-II

Öğr. Gör. Erhan AKAGÜNDÜZ

HATA AYIKLAMA VE HATA KONTROLÜ

- **Yazılım Geliştirme Süreci:**
  - Kod yazarken beklenmedik sonuçlar ortaya çıkabilir.
- **Hata Ayıklama:**
  - Hataların kaynağını tespit etmek için yapılan işlemdir.
- **Araçlar:**
  - Visual Studio gibi programlar, kodu adım adım çalıştırarak hataları bulmayı sağlar.

HATA AYIKLAMA VE HATA KONTROLÜ

- **Geçici Değişiklikler:**
  - Hata ayıklama sırasında düzeltmeleri gözlemleyebiliriz.
- **Hata Kontrolü:**
  - Kullanıcı hatalarını önlemek için sistemde kontrol mekanizmaları kurmak önemlidir.

HATA AYIKLAMA ARAÇLARI

- **Hata Nedenleri:** Hatalar genellikle kullanıcı kaynaklıdır. Örneğin, metin alanına sayı girilmesi gibi. Yazılımcılar, bu tür hataları önlemek için kod yazarlar.

HATA AYIKLAMA ARAÇLARI

- **Hata Türleri:**
- **Yazılım Hataları:** Yazım hataları; örneğin, **Console.WriteLine** yerine **ConsoleWriteLine** yazmak.
- **Çalışma Zamanı Hataları:** Derleme sırasında oluşan hatalar; ondalıklı sayılarla yapılan işlemler.
- **Mantıksal Hatalar:** Sistemin çalışmasına rağmen yanlış sonuçlar üreten hatalar; negatif değerlerin kullanılması gibi.

DEBUG MENÜSÜ ELEMANLARI

- **Start (F5):** Uygulamayı başlatır.
- **Restart (Ctrl+Shift+F5):** Debug modunu sonlandırır ve yeniden başlatır.
- **Stop Debugging (Shift+F5):** Hata ayıklamayı durdurur.
- **Step Into (F11):** Satır satır kod çalıştırır; metot çağırıldığında o metot içerisinde durur.
- **Step Over (F10):** Bir metot çağırıldığında, önce metodu çalıştırır, sonra devam eder.
- **Step Out:** Çalışan metot içerisindeki tüm komutları çalıştırır ve bir sonraki satıra geçer.
- **Watch:** Değişkenlerin değerlerini izlemek için kullanılır; sağ tıklayıp **"Add Watch"** ile eklenir.

HATA AYIKLAMA ARAÇLARI

- **TaskList (Görev Listesi):**
  - Tanımlanmayan fonksiyon, değişken ve yazım hatalarını gösterir.
  - **Açılış:** Project -> Other Windows -> Task List veya Ctrl+Alt+K ile açılabilir.
- **Output Penceresi**
  - Derleme aşamalarını gösterir.
  - **Açılış:** Ctrl+Alt+O ile açılır.
  - Derleme, hata ayıklama ve paket yöneticisi gibi seçenekler içerir.

HATA AYIKLAMA ARAÇLARI

- **Breakpoint (Kesme Noktası):**
  - Kodda belirli bir satırda duraklamak için kullanılır.
  - **Ekleme:** F9 tuşu veya sağ tıkla "Insert Breakpoint" seçeneği ile eklenir.
  - **Koşullar:**
    - Belirli bir sayıda durdurma.
    - Sayının katlarında durdurma.
    - Her breakpoint'e geldiğinde durma.
    - Belirli bir sayı ve üzerindeki sayılar için durma.

HATA AYIKLAMA VE İZ BULMA SINIFLARI

- **Trace ve Debug Sınıfları:**
  - Hata mesajları ve dosyalama bilgilerini yönetmek için kullanılır.
  - **Kullanım:** *using System.Diagnostics;* ile başlatılır.

```
Trace.WriteLine("Trace sınıfı Yazdırıldı");
Debug.WriteLine("Debug sınıfı Yazdırıldı");
```

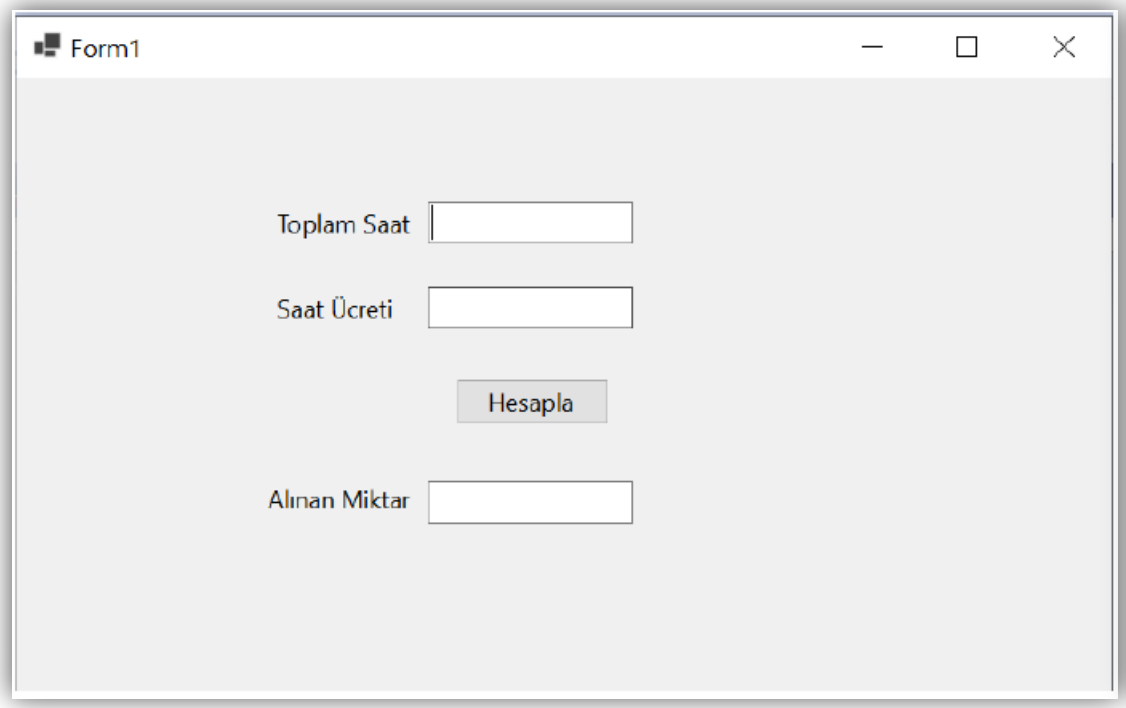
İSTİSNAİ DURUMLAR

```
try {
    // Hata olabilecek kodlar
} catch {
    // Hata durumunda çalışacak kodlar
}
```

- **System.Exception Sınıfı**
  - Tüm istisnai durum sınıflarının temel sınıfıdır.
  - Alt Sınıflar: SystemException ve ApplicationException olarak iki ana sınıfa ayrılır.
- **Try-Catch Kullanımı**
  - Hataları kontrol etmek için kullanılır.
  - Try bloğunda hata olabilecek kodlar, Catch bloğunda hata durumunda çalışacak kodlar bulunur.

ÖRNEK

- Catch kullanılmadan finally bloğunu kullanabileceğimize dair örnek.




```
private void button1_Click(object sender, EventArgs e) {
    try {
        int toplamsaat = int.Parse(textBox1.Text);
        int saatucreti = int.Parse(textBox2.Text);
        textBox3.Text = Convert.ToString(toplamsaat * saatucreti);
    } finally {
        textBox1.Text = "";
        textBox2.Text = "";
        textBox1.Focus();
    }
}
```

THROW VE RETHROW KULLANIMI

- **Throw Kullanımı**
  - Hataları bilinçli olarak oluşturmak için kullanılır.
  - Kullanıcı gerekli alanları doldurmazsa, hata mesajı gösterilir.

```
private void button1_Click(object sender, EventArgs e) {
    string adi = textBox1.Text;
    string soyadi = textBox2.Text;
    string ogrno = textBox3.Text;

    if (adi == "" || soyadi == "" || ogrno == "") {
        try {
            throw new Exception();
        } catch {
            MessageBox.Show("Lütfen alanları eksiksiz doldurunuz",
                "Dikkat!");
        }
    } else {
        MessageBox.Show("Tüm bilgiler girildi!");
    }
}
```

THROW VE RETHROW KULLANIMI

```
class RethrowKullanimi {
    public static void Islemleriyurut() {
        int[] ilkDizi = { 5, 25, 125, 225, 128, 256, 521, 1048 };
        int[] ikinciDizi = { 1, 5, 0, 2, 4, 3 };

        for (int j = 0; j < ilkDizi.Length; j++) {
            try {
                Console.WriteLine(ilkDizi[j] + "/" + ikinciDizi[j] + "=" + ilkDizi[j] / ikinciDizi[j]);
            } catch (DivideByZeroException) {
                Console.WriteLine("Sıfıra Bölünmez!");
            } catch (IndexOutOfRangeException) {
                Console.WriteLine("Eşleşen Eleman Bulunamadı!");
                throw;  // Hata yeniden fırlatılır
            }
        }
    }
}

static void Main(string[] args) {
    try {
        RethrowKullanimi.Islemleriyurut();
    } catch (IndexOutOfRangeException) {
        Console.WriteLine("Kritik Hata!");
        throw;  // Hata yeniden fırlatılır
    }
}
```

THROW VE RETHROW KULLANIMI

